אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Applied Deep Learning

Omri Azencot

The Stein Faculty of Computer and Information Science

Ben-Gurion University of the Negev

Deep Feedforward Networks

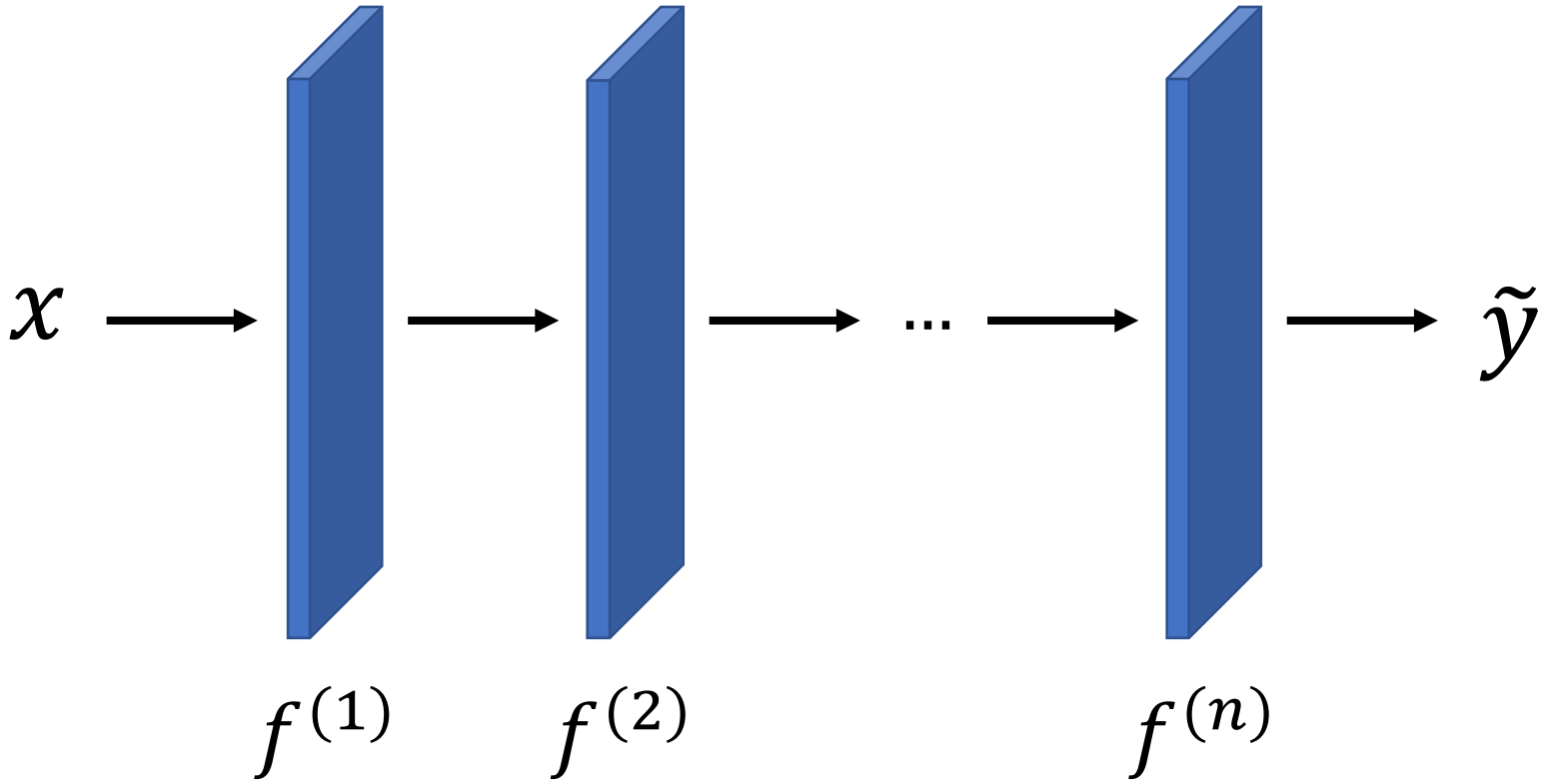
Feedforward Neural Networks

x

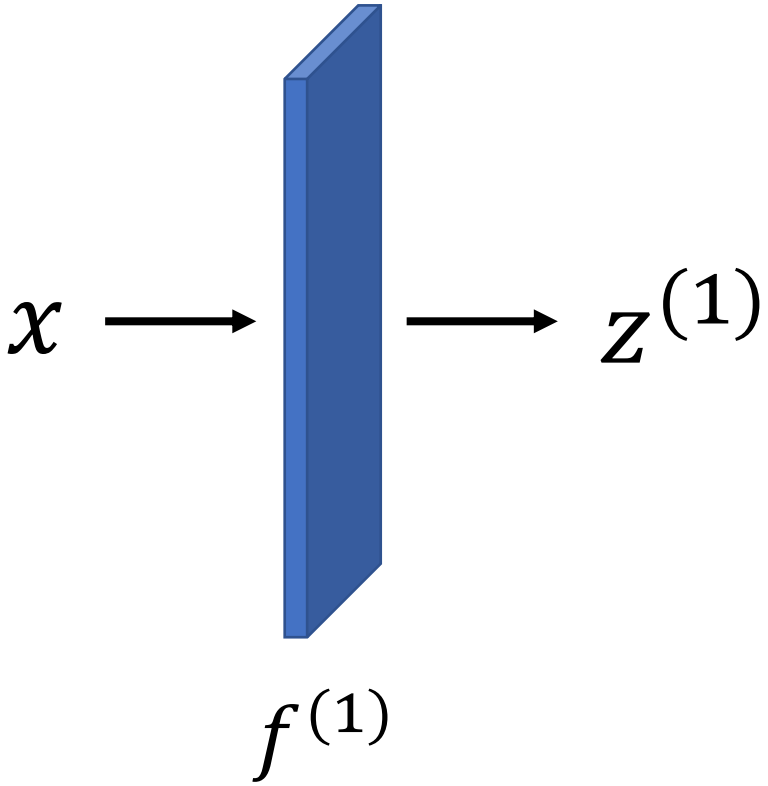
$f(x;\theta)$

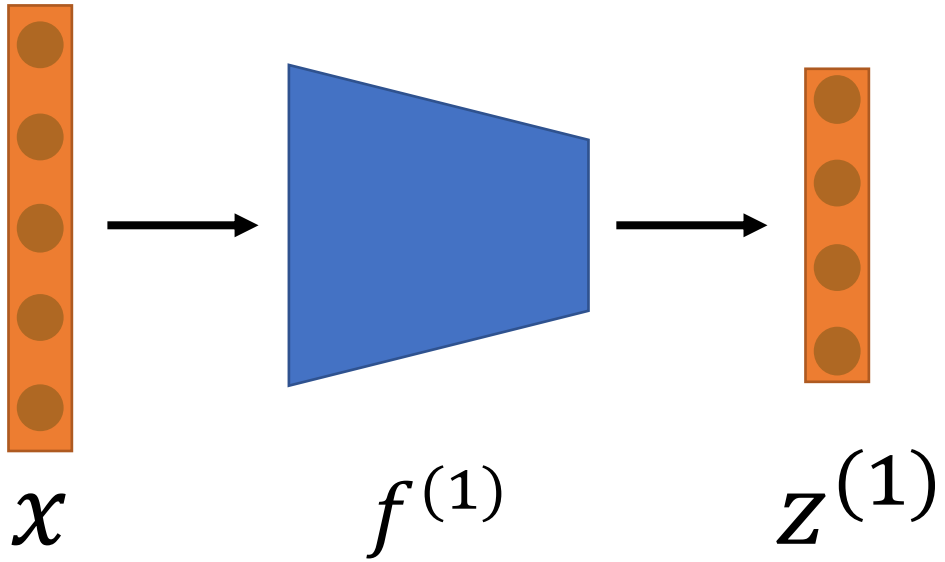
CAT

Feedforward Neural Networks



Feedforward Neural Networks





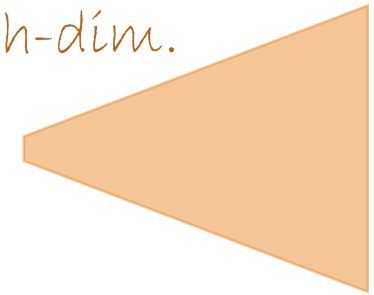
How to model $f(x;\theta)$?

Recall the linear model

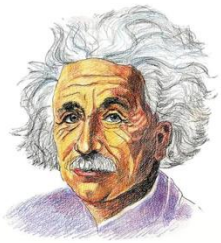
$$f(x;\theta) = x^T\theta := \tilde{y}$$ ← limited to linear relations

What about a nonlinear transform of x, i.e., $\phi(x)$? How to choose ϕ?

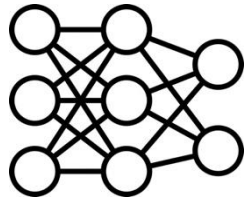
high-dim.



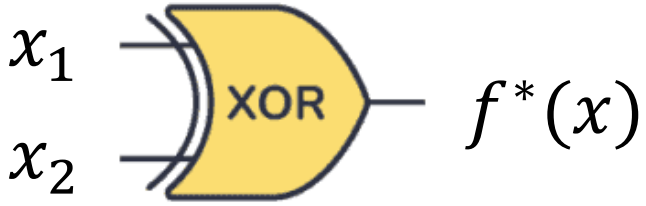
manual design



neural networks



Example: Learning XOR



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

$$\mathbb{X} = \{[0,0]^T, [1,0]^T, [0,1]^T, [1,1]^T\} \quad \textit{data}$$

$$\mathcal{J}(\theta) = \frac{1}{4}\sum_{x\in\mathbb{X}}\left(f^*(x) - f(x;\theta)\right)^2 \quad \textit{loss}$$

A Linear Model for XOR

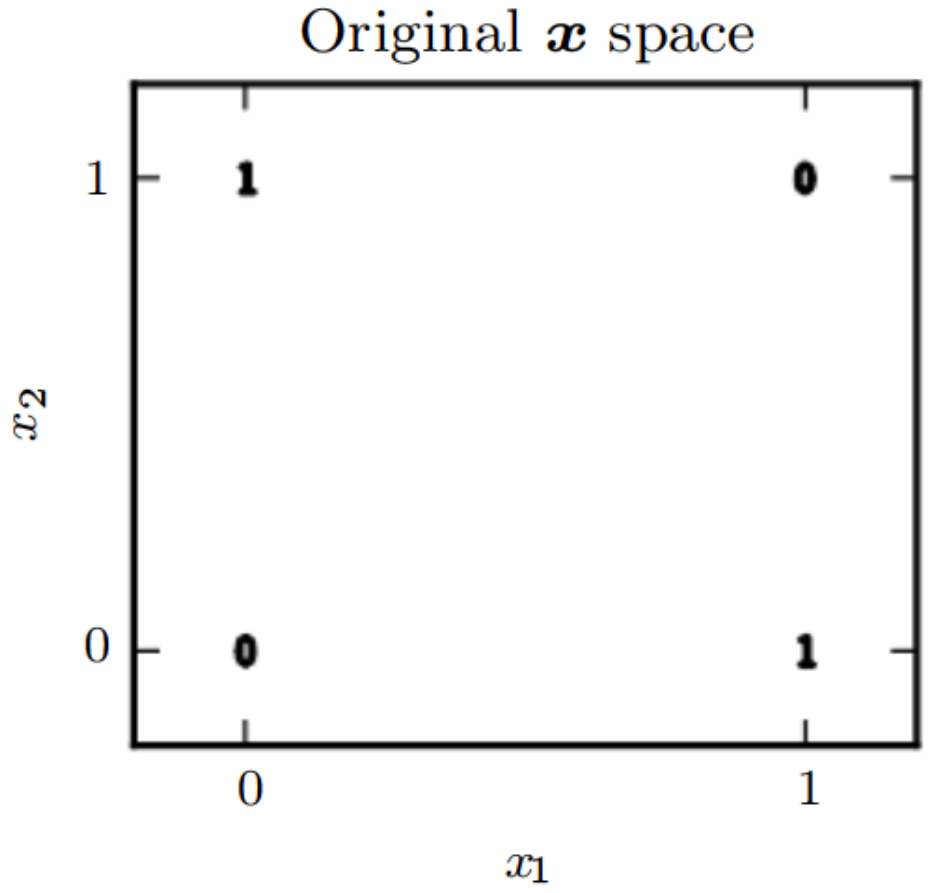
We consider the linear model

$$f(x;\omega,b) = x^T\omega + b$$

Minimizing $\mathcal{J}(\theta)$ yields

$$\omega = 0, b = \frac{1}{2}$$

why?



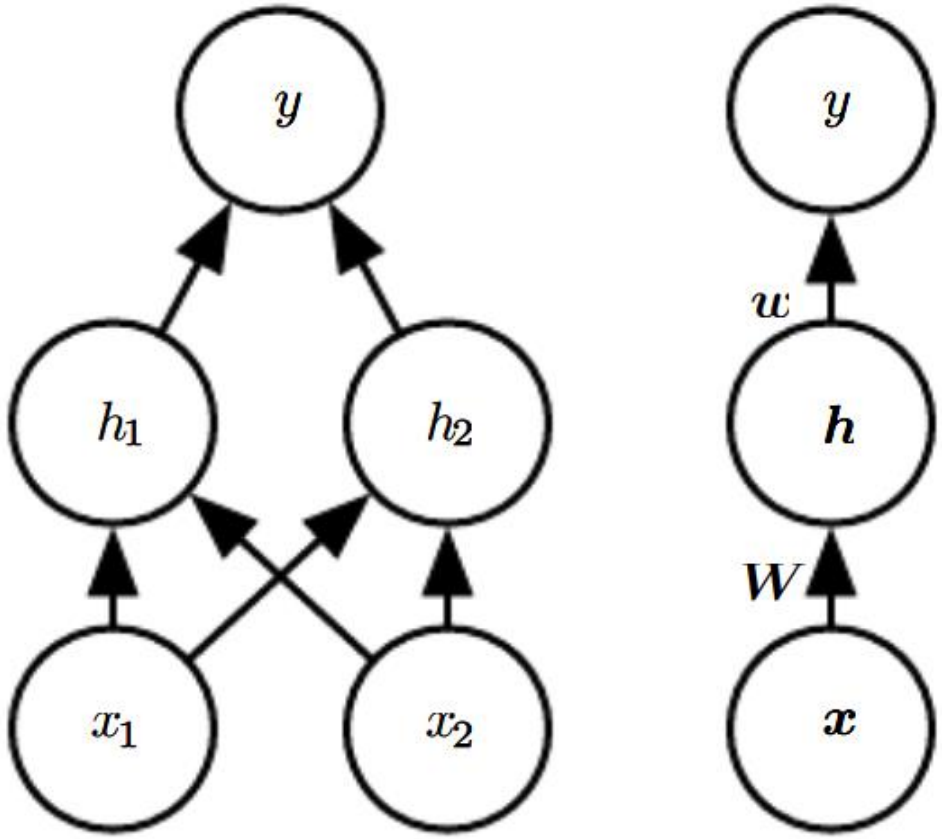
A Feedforward NN for XOR

Consider the NN

$$f(x; W, c, \omega, b) = f^{(2)}(f^{(1)}(x))$$

What form $f^{(1)}$ takes?

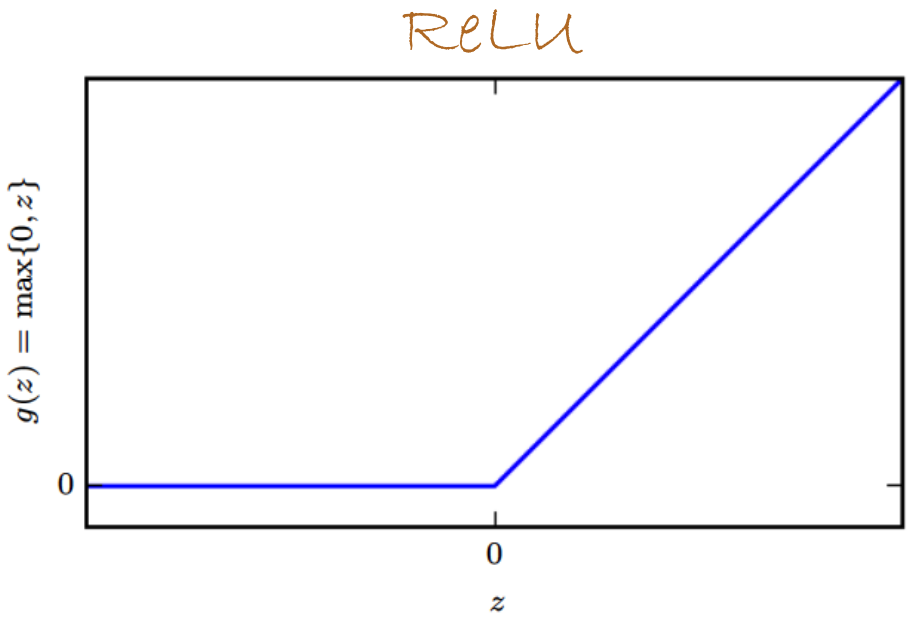
It can not be linear!

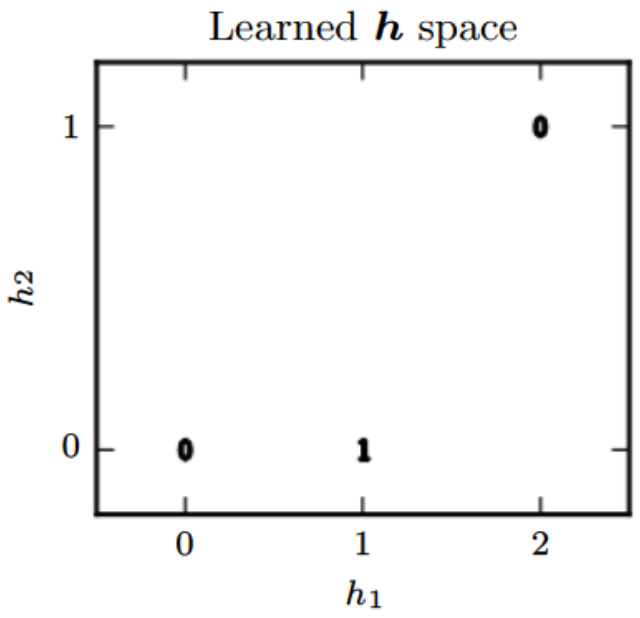


A Feedforward NN for XOR

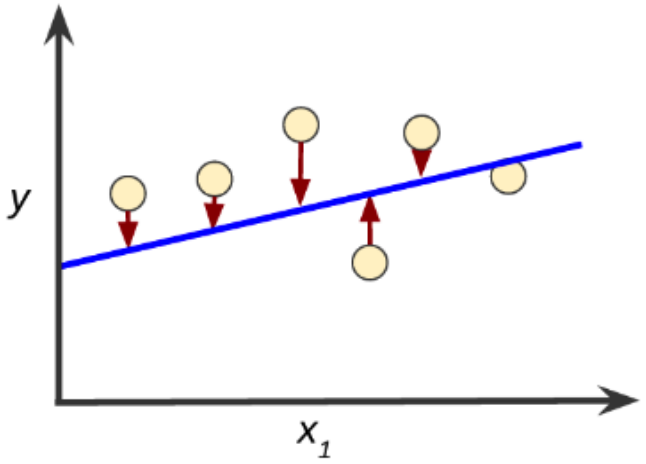
We use a nonlinear $f^{(1)}$!

$$f(x; W, c, \omega, b) = \omega^T \mathrm{ReLU}(W^T x + c) + b$$

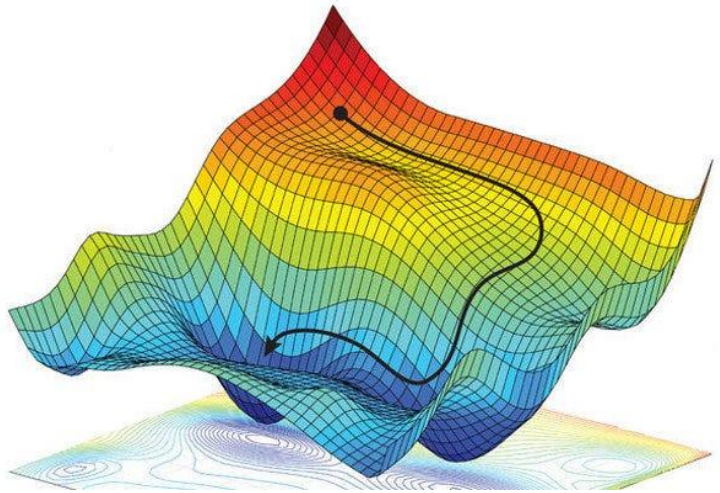




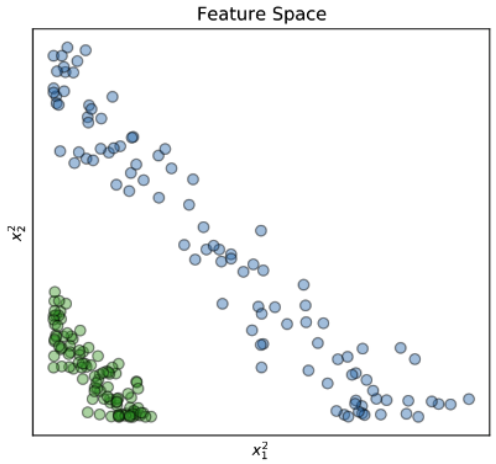
Machine Learning Models



loss function



optimizer



output representation

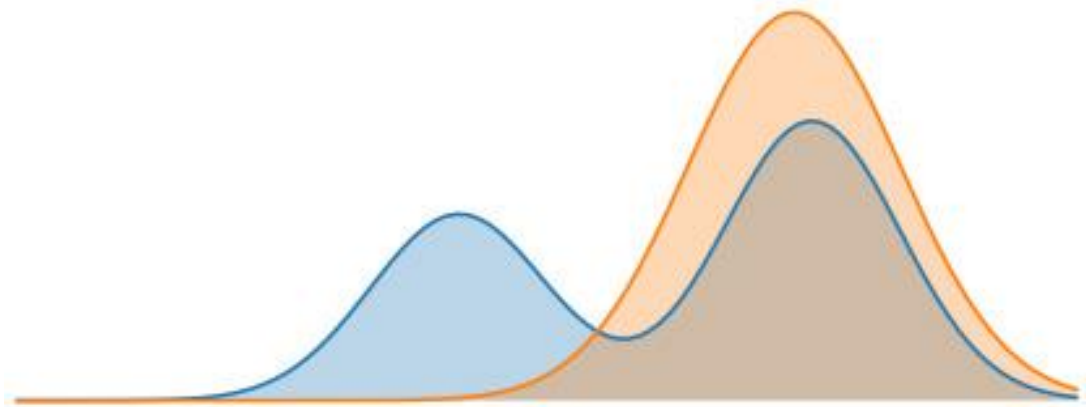
Maximum Likelihood Estimation

How to design good estimators?

$$\theta_{\mathrm{ML}} = \arg\max_{\theta} p_{\mathrm{model}}(\mathbb{X}; \theta)$$

$$\theta_{\mathrm{ML}} = \arg\max_{\theta} \mathbb{E}_{\mathrm{x}\sim\hat{p}_{\mathrm{data}}} \log p_{\mathrm{model}}(x; \theta)$$

(negative) log-liklihood



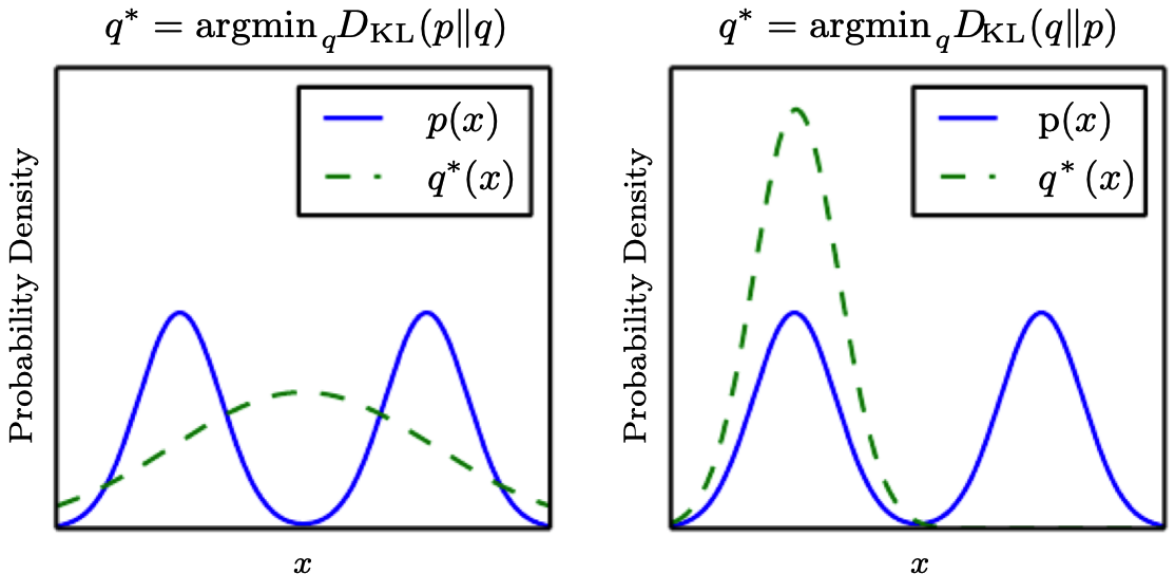
KL Divergence

Distance between Distributions

Kullback—Leibler divergence

$$D_{KL}(P\|Q) = \mathbb{E}_{x\sim P}\left[\log\frac{P(x)}{Q(x)}\right]$$

asymmetric



Maximum Likelihood Estimation (MLE)

MLE for θ is

$$\theta^* = \arg\max_\theta p_{\text{model}}(\mathbb{X}; \theta)$$

model data

Related to KL distance

$$D_{\text{KL}}(\hat{p}_{\text{data}} \parallel p_{\text{model}}) = \mathbb{E}_{x\sim\hat{p}_{\text{data}}}[\log \hat{p}_{\text{data}}(x) - \log p_{\text{model}}(x)]$$

train set

Equiv. to

$$\theta^* = \arg\max_\theta \mathbb{E}_{x\sim\hat{p}_{data}} \log p_{\text{model}}(x; \theta)$$

Equiv. to

cross entropy

$$-\mathbb{E}_{x\sim\hat{p}_{\text{data}}}[\log p_{\text{model}}(x)]$$

Example: Linear Regression as MLE

Supervised learning uses

$$\theta^* = \arg\max_\theta P(Y|X;\theta)$$

If samples i.i.d

$$\theta^* = \arg\max_\theta \sum_i \log p(y_i|x_i;\theta)$$

Define

$$p(y|x) = \mathcal{N}(y;\hat{y}(x;\omega),\sigma^2)$$

The conditional log-likelihood

$$\sum_i \log p(y_i|x_i;\theta) = -m\log\sigma - \frac{m}{2}\log(2\pi) - \sum_i \frac{|\hat{y}_i - y_i|^2}{2\sigma^2}$$

Output Units: Gaussian Distributions

For regression problems, the model predicts

$$\hat{y} = W^T h + b$$

The underlying assumption is that

$$p(y|x) = \mathcal{N}(y; \hat{y}, I)$$

Output Units: Bernoulli Distributions

For binary classification problems, we want to estimate for $y \in \{0,1\}$

$$P(y = 1|x)$$

A Bernoulli random variable has probability:

$$P(y) = p^y(1-p)^{1-y}$$

where $p = P(y = 1), 1 - p = P(y = 0)$

Output Units: Bernoulli Distributions

For binary classification problems, the model predicts

$$z = W^T h + b$$

A naïve approach would be:

$$\hat{y} = \max\{0, \min\{1, z\}\}$$

Why is this problematic?

Output Units: Bernoulli Distributions

For binary classification problems, the model predicts

$$\hat{y} = \sigma(W^T h + b) \longleftarrow \text{sigmoid}$$

Therefore

$$P(y = 1|x) = \sigma(z), P(y = 0|x) = 1 - \sigma(z)$$

Output Units: Bernoulli Distributions

We have

$$P(y = 1|x) = \sigma(z), P(y = 0|x) = 1 - \sigma(z)$$

We now train the network using MLE

$$\max \log P(y|x) \qquad \text{or} \qquad \min -\log P(y|x)$$

After simplification: $J(\theta) = -yz + \log(1 + e^z)$ ← Binary cross entropy loss

Output Units

Gaussian distributions

$$\hat{y} = W^T h + b$$

hidden

$$p(y|x) = \mathcal{N}(y; \hat{y}, I)$$

Bernoulli distributions

$$\hat{y} = \sigma(W^T h + b)$$

$$\mathcal{J}(\theta) = -yz + \log(1 + e^z)$$

Multinoulli distributions

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum \exp(z_j)}$$

ξ

$$\log \xi(z)_i = z_i - \log \sum \exp(z_j)$$

Hidden Units

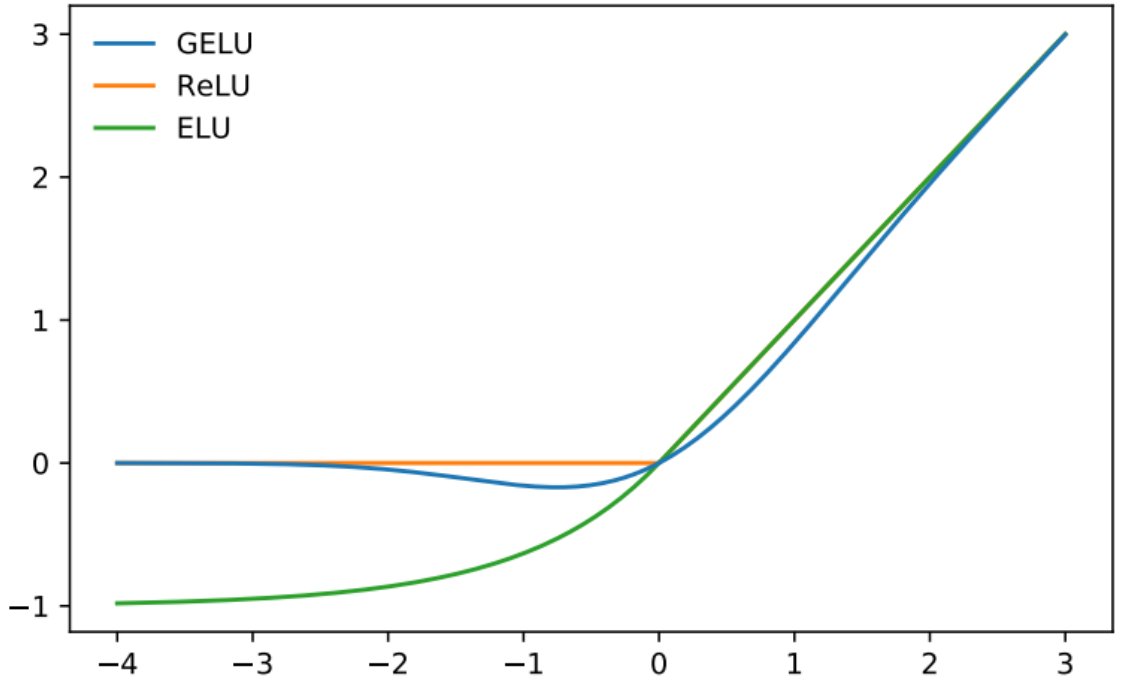
Selecting the type of the hidden unit is an active research area..

In most cases, hidden units are defined via

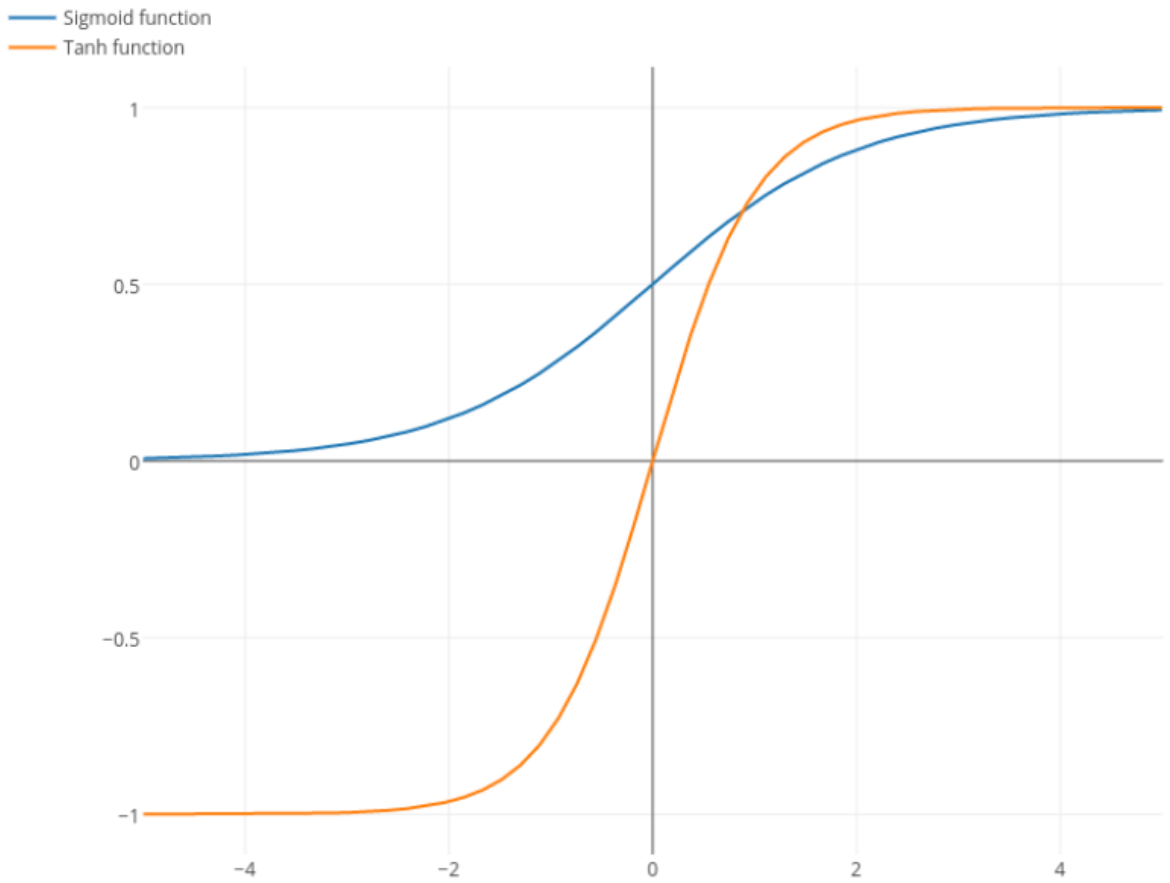
$$z = W^T x + b \Rightarrow h = g(z)$$

where $g(\cdot)$ is an activation function

Rectified Linear Units (ReLU)

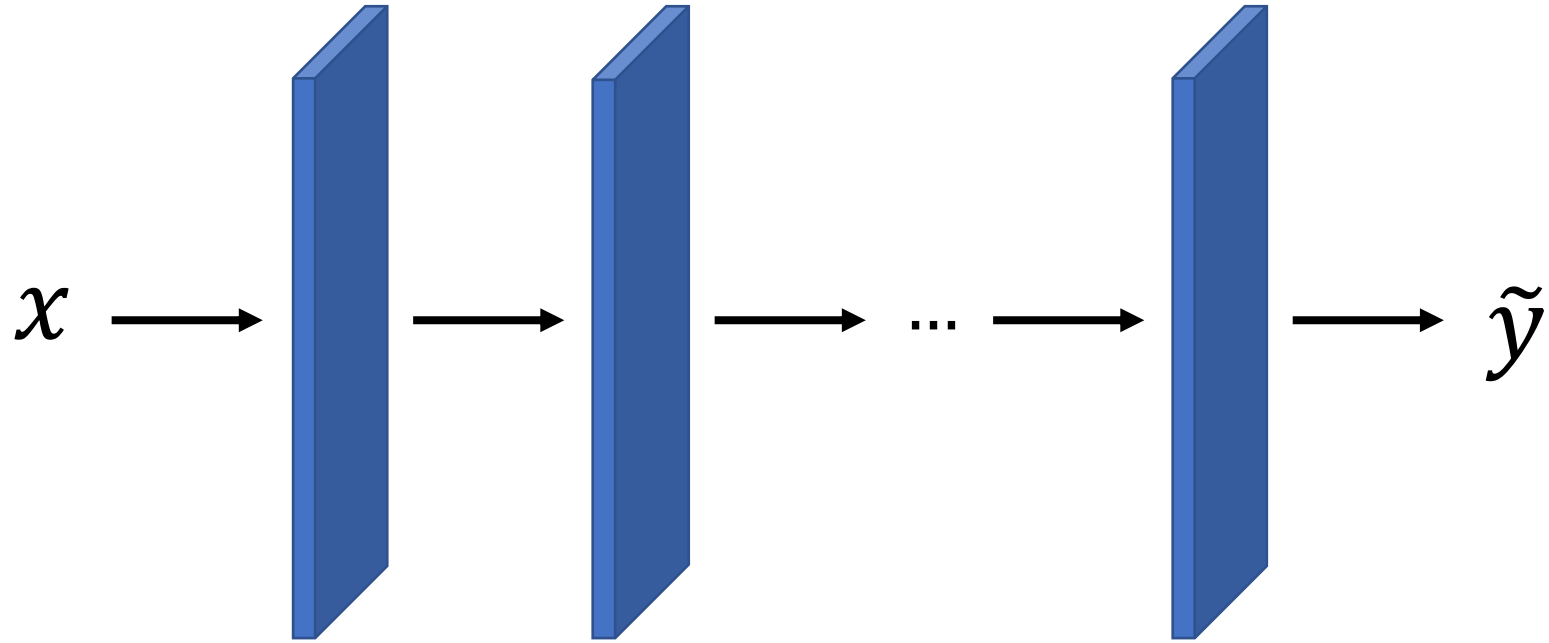


Logistic Sigmoid and Hyperbolic Tangent



Architecture Design

How to decide about the overall structure of the network?



chain structure

Architecture Design

How to decide about the overall structure of the network?

first layer $h^{(1)} = g^{(1)}(W^{(1)T}x + b^{(1)})$

second layer $h^{(2)} = g^{(2)}(W^{(2)T}h^{(1)} + b^{(2)})$

Decide about the depth and width of the network

The Universal Approximation Theorem

Let $\epsilon > 0$, then there is $r(W) > N$ such that the feedforward network

#rows

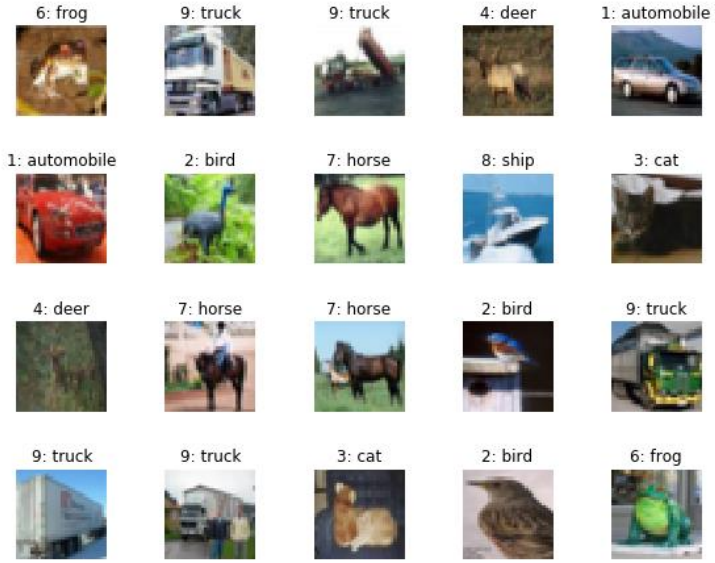
$$y = U^T \sigma(W^T x + b) + c$$

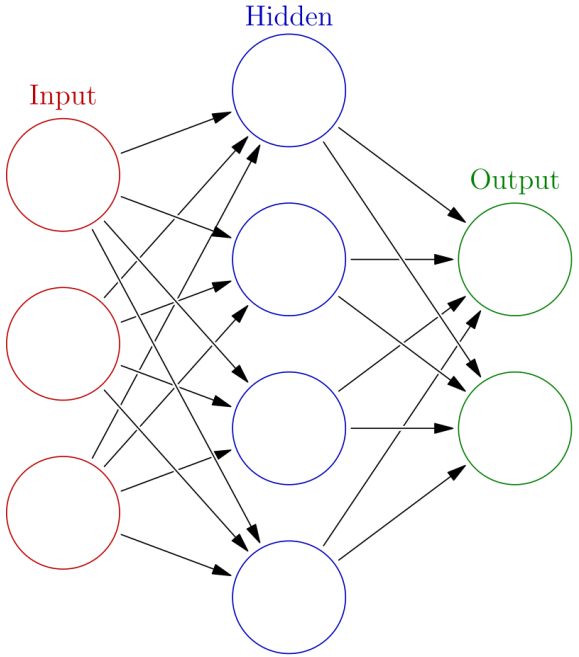
"squashing"

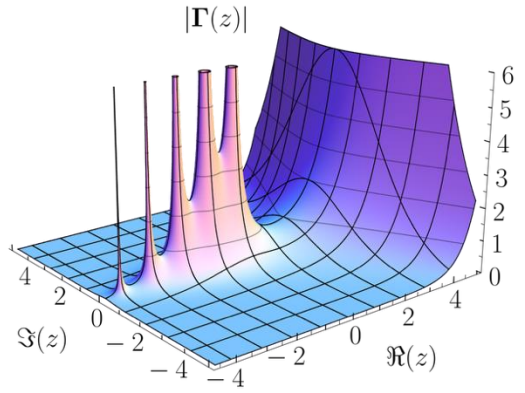
satisfies $|f(x) - y| < \epsilon$ where $f: \mathbb{R}^m \to \mathbb{R}^n$ is Borel measureable.

continuous on a closed and bounded subset of $\mathbb{R}^m$

Universal Approximation



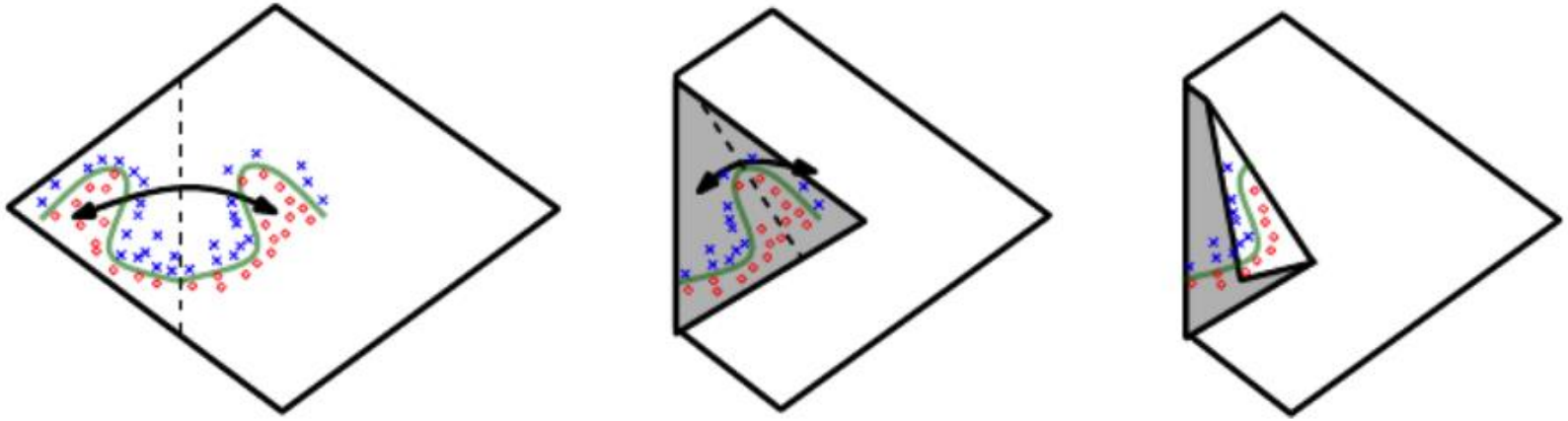




Width for Universal Approximation

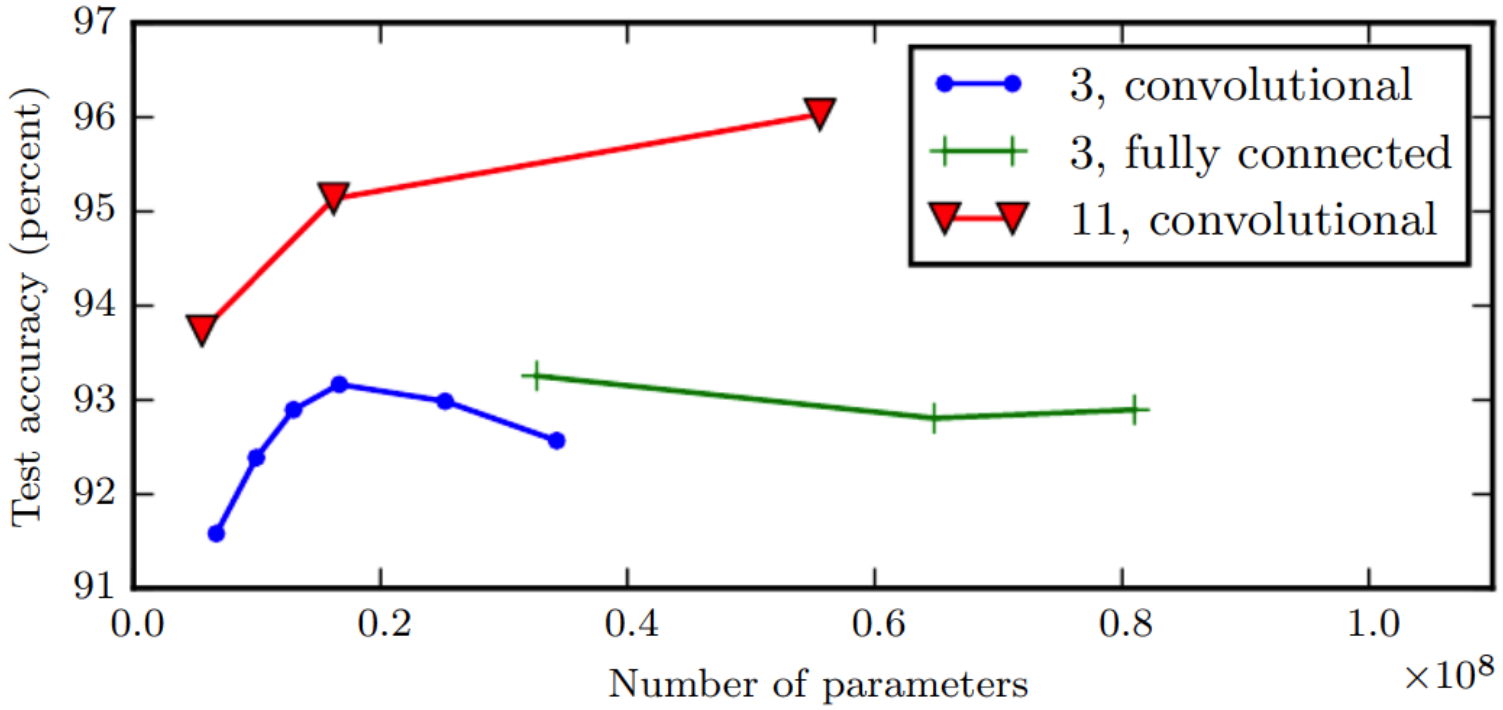
Montufar et al. ‘14:

shallow rectifier nets require an exponential number of hidden units to approximate functions



Deep Networks

Depth helps better than width!



Code Demo